Instituto Federal Catarinense

Bacharelado em Ciência da Computação

Campus Rio do Sul



Laila Vitória Marconatto

Comparação entre estruturas de dados

Trabalho da disciplina de Estrutura de Dados 2
Professor Daniel Gomes Soares

Rio do Sul

2026

1 INTRODUÇÃO

Este trabalho compara o desempenho de sete estruturas de dados implementadas em Java: array linear, array ordenado, lista duplamente encadeada (chamada de lista dupla nas tabelas), tabela hash, árvore binária de busca sem balanceamento (BST), árvore AVL e árvore rubro-negra. O objetivo é observar como o tamanho do conjunto e a ordem de inserção influenciam construção, buscas, inserção, remoção e consultas por intervalo.

Os resultados mostram que a escolha depende da operação e da implementação. O array ordenado apresentou as menores medianas nas consultas por intervalo em todos os cenários. A BST teve forte degradação nas entradas ordenadas, enquanto AVL e rubro-negra mantiveram custos menores. A hash teve os menores tempos nas buscas exatas e atualizações com 100 mil elementos. Esse conjunto, porém, produziu um arranjo final sem colisões, especialmente favorável à função utilizada; os resultados não permitem generalizar essa vantagem.

2 DESENVOLVIMENTO E ARQUIVOS DO PROJETO

O código-fonte, os arquivos CSV e este relatório estão disponíveis no repositório LailaMarconatto/trabalho1_estruturaDados2, no GitHub.

As estruturas foram adaptadas dos códigos das aulas. A interface Estrutura padroniza inserir, buscar, remover, contarIntervalo e tamanho. Inserções repetidas são ignoradas, e inserção e remoção retornam se houve alteração. Os arrays foram implementados para a comparação; nenhuma coleção pronta substitui as estruturas avaliadas.

A classe Testes verifica operações, duplicatas, intervalos e invariantes das árvores balanceadas. A classe Experimento prepara os lotes, reconstrói estados independentes, cronometra as operações e grava resultados_brutos.csv e resumo.csv. As adaptações estão descritas em ALTERACOES.md; os originais preservados ficam em referencia-aulas e as versões experimentais em src.

3 AMBIENTE DE TESTES

Item	Configuração registrada
Processador	Intel Core i7-8550U CPU @ 1,80 GHz
Memória física utilizável	15,89 GiB informados pelo Windows
Sistema operacional	Windows 11 amd64
Java	OpenJDK 21.0.10, 64-Bit Server VM
Execução	Aplicação Java por PowerShell, sem depurador
Conclusão da rodada	4 de novembro de 2026

A memória acima é a quantidade utilizável pelo sistema, e não o limite do heap da JVM. O ambiente completo está registrado em resultados/ambiente.txt.

3.1 Metodologia de medição

Foram utilizados conjuntos iniciais de 5.000, 10.000 e 100.000 valores pares de 2 até 2n, inseridos em ordem crescente, decrescente ou aleatória. Os sorteios usam semente 42. Para cada tamanho e ordem, todas as estruturas e repetições recebem os mesmos dados e lotes de operações.

Teste	Operações em cada lote
Construção	Criar a estrutura e inserir n valores, um a um
Busca presente	2.000 buscas por pares do conjunto inicial
Busca ausente	2.000 buscas por ímpares entre 1 e 2n − 1
Inserção	Inserir n/10 ímpares distintos
Remoção	Remover n/10 pares distintos, localizando-os internamente
Intervalo	200 mínimos sorteados entre 1 e 2n − 100; máximo = mínimo + 100

Cada combinação teve um aquecimento e cinco execuções medidas com System.nanoTime(). A estrutura é reconstruída antes de toda execução. Essa preparação fica fora do trecho cronometrado, exceto na construção, que inclui a criação da estrutura. Sorteios, conferências, mensagens e escrita de arquivos também ficam fora do tempo medido.

O tempo total de cada lote foi armazenado em nanossegundos. A mediana corresponde ao terceiro dos cinco tempos ordenados. No resumo, o total é convertido para milissegundos e o tempo por operação é obtido dividindo a mediana pelo número de operações do lote. As tabelas arredondam os números somente para apresentação; os CSVs preservam a precisão registrada.

A rodada gerou 1.890 medições e 378 resumos. O programa conferiu 2.000 acertos nas buscas presentes, zero acertos nas ausentes, tamanhos finais de 1,1n e 0,9n após inserção e remoção e somas de intervalos iguais às esperadas em todas as estruturas. Testes verifica as estruturas; validar_resultados.py confere a cobertura e a aritmética dos CSVs, não a correção dos algoritmos.

3.2 Características das implementações

Os arrays reservam n mais 10% de n posições. O array linear remove substituindo o valor pelo último elemento, mas a busca pelo valor continua sequencial. O array ordenado usa busca binária e deslocamentos com System.arraycopy; sua construção mantém a ordenação a cada inserção. A lista insere ao final, após procurar duplicatas.

A hash usa sondagem linear, capacidade inicial 16 e duplicação ao superar carga de 0,7, considerando posições ocupadas ou marcadas como removidas. A função é floorMod(valor, capacidade). A AVL usa inserção e remoção recursivas e armazena alturas. A rubro-negra usa percursos e correções iterativas, ponteiros para o pai e uma sentinela preta. Ambas rebalanceiam após atualizações; os custos refletem também essas diferenças de implementação. A BST não é balanceada e utiliza percursos iterativos para evitar estouro de pilha em entradas grandes ordenadas.

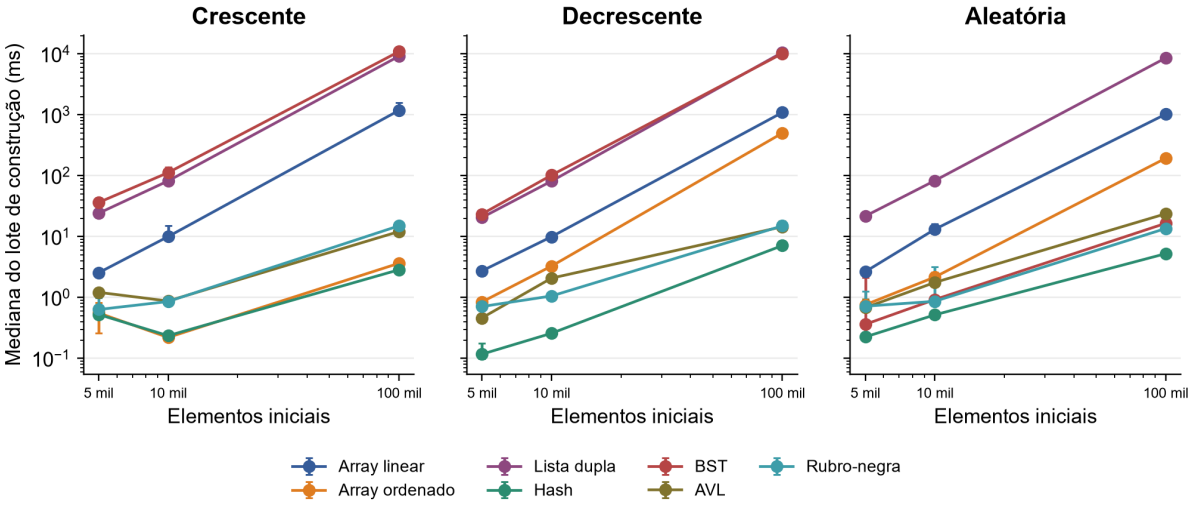
4 ANÁLISE COMPARATIVA ENTRE ESTRUTURAS

4.1 Construção e visão geral

Tabela 1 Entrada aleatória com 100.000 elementos

Estrutura	Constr. ms	Busca pres. ns/op	Busca aus. ns/op	Inserção ns/op	Remoção ns/op	Intervalo ns/op
Array linear	1.022,72	8.691,70	20.195,30	22.041,51	9.359,39	200.644,50
Array ordenado	193,30	102,55	110,20	4.463,62	3.720,19	223,50
Lista dupla	8.581,10	94.700,00	191.379,90	178.945,87	83.607,45	351.576,50
Hash	5,24	10,90	7,70	13,72	10,03	110.878,00
BST	16,51	172,10	182,95	187,67	152,13	1.239,50
AVL	23,67	98,30	137,60	301,90	299,22	944,00
Rubro-negra	13,56	93,90	109,05	162,75	166,55	914,50

Figura 1 Tempo total de construção nas três ordens de entrada



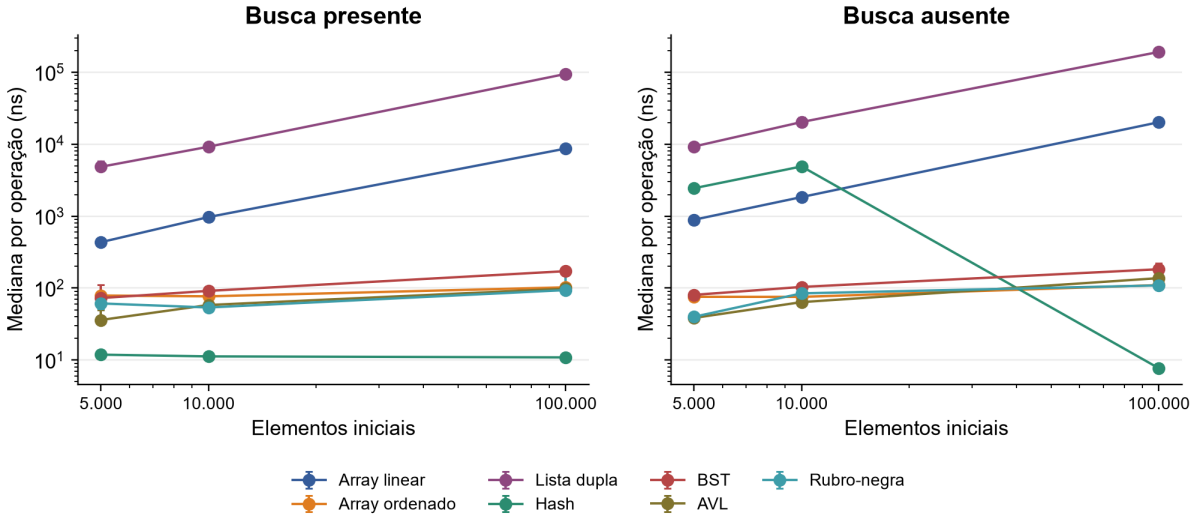
Fonte: CSVs do experimento. Pontos: medianas; barras: mínimo e máximo das cinco medições, não intervalos de confiança. Eixos de tempo logarítmicos.

Com 100 mil elementos aleatórios, a hash teve a menor mediana de construção, 5,24 ms, e a lista a maior, 8.581,10 ms. A lista e o array linear precisam procurar duplicatas a cada inserção. Por isso, construir o conjunto inteiro tem custo quadrático, mesmo que acrescentar um elemento no final seja simples.

No array ordenado, a entrada crescente custou 3,64 ms, a aleatória 193,30 ms e a decrescente 500,29 ms. A ordem crescente evita deslocamentos, enquanto a decrescente desloca todos os elementos já armazenados em cada inserção. Isso explica a diferença sem mudar a regra de manter o array ordenado durante a construção.

4.2 Impacto da escala e comportamento das buscas

Figura 2 Buscas por valores presentes e ausentes em entrada aleatória



Fonte: CSVs do experimento. Pontos: medianas; barras: mínimo e máximo das cinco medições, não intervalos de confiança. Eixos de tempo logarítmicos.

Tabela 2 Aumento do tempo por operação de 5.000 para 100.000 elementos aleatórios

Estrutura	Busca pres. fator	Busca aus. fator	Intervalo fator	Referência buscas
Array linear	19,99×	22,65×	20,08×	20× linear
Array ordenado	1,30×	1,46×	1,45×	1,35× log
Lista dupla	19,51×	20,61×	20,16×	20× linear
Hash	0,92×	0,0032×	29,06×	1× ideal
BST	2,37×	2,27×	1,72×	1,35× médio
AVL	2,72×	3,57×	1,93×	1,35× log
Rubro-negra	1,53×	2,73×	1,72×	1,35× log

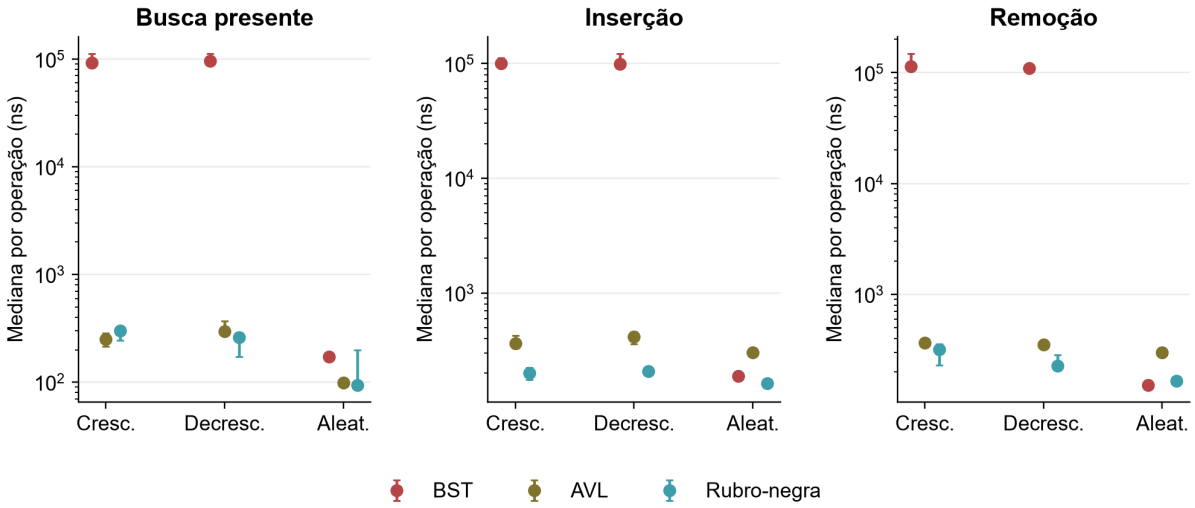
O conjunto aumentou 20 vezes. A busca presente do array linear cresceu de 434,85 para 8.691,70 ns/op, aproximadamente 19,99 vezes. Na lista, passou de 4.852,95 para 94.700,00 ns/op, cerca de 19,51 vezes. Esses resultados são compatíveis com o percurso linear. Na entrada aleatória, o lote de construção do array linear cresceu cerca de 386 vezes e o da lista 396 vezes, próximos ao fator 400 esperado para crescimento quadrático quando n aumenta 20 vezes.

O array ordenado passou de 78,90 para 102,55 ns na busca presente. A AVL passou de 36,10 para 98,30 ns e a rubro-negra de 61,25 para 93,90 ns. O modelo puramente logarítmico prevê aproximadamente 1,35 vez para esse aumento de n. A AVL cresceu 2,72 vezes na busca presente e 3,57 vezes na ausente, acima dessa referência. O limite assintótico permanece logarítmico, mas não explica sozinho as constantes observadas; cache e JVM são hipóteses não isoladas nesta rodada.

Com 100 mil elementos aleatórios, a busca ausente custou 2,32 vezes a presente no array linear e 2,02 vezes na lista. A busca presente pode parar ao encontrar o valor; a ausente precisa percorrer o conjunto inteiro. A diferença não é obrigatoriamente o dobro. No array ordenado, foram 102,55 e 110,20 ns/op, respectivamente.

4.3 Impacto da ordem de inserção nas árvores

Figura 3 Operações nas árvores com 100.000 elementos por ordem de entrada



Fonte: CSVs do experimento. Pontos: medianas; barras: mínimo e máximo das cinco medições, não intervalos de confiança. Eixos de tempo logarítmicos.

Tabela 3 Construção e busca presente nas árvores com 100.000 elementos

Estrutura e ordem	Construção ms	Busca presente ns/op
BST crescente	10.917,56	92.371,60
BST decrescente	10.150,54	96.265,90
BST aleatória	16,51	172,10
AVL crescente	12,09	250,55
AVL decrescente	14,48	296,50
AVL aleatória	23,67	98,30
Rubro-negra crescente	15,01	299,80
Rubro-negra decrescente	14,98	262,00
Rubro-negra aleatória	13,56	93,90

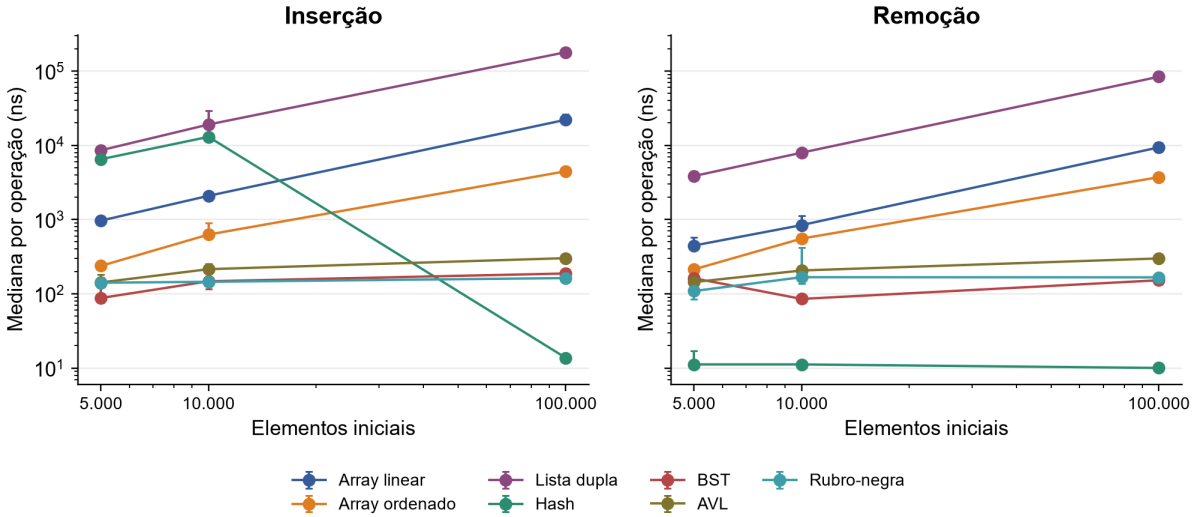
A BST levou 16,51 ms para construir a entrada aleatória, 10.917,56 ms na crescente e 10.150,54 ms na decrescente. A construção crescente ficou cerca de 661 vezes mais lenta que a aleatória. As entradas ordenadas geram uma cadeia com altura proporcional a n, levando a operações lineares e construção quadrática. A implementação iterativa permitiu medir os 100 mil valores sem substituir esse cenário por um conjunto menor.

A degradação da BST já aparece nos tamanhos menores: para 5 mil, a construção aleatória custou 0,36 ms, contra 36,22 ms na crescente e 23,42 ms na decrescente; para 10 mil, foram 0,92 ms, 112,12 ms e 101,78 ms. Os valores exatos estão nos apêndices, sem misturar tamanhos nas comparações.

AVL e rubro-negra mantiveram construção entre 12,09 e 23,67 ms nas três ordens. Elas não foram imunes à ordem: a busca presente da AVL variou de 98,30 a 296,50 ns/op, e a da rubro-negra de 93,90 a 299,80 ns/op. O balanceamento evita a degeneração da BST. A causa das diferenças entre ordens nas árvores balanceadas não foi estabelecida: não foram registradas profundidades médias nem eventos da JVM ou do hardware.

4.4 Inserções e remoções

Figura 4 Inserção e remoção por operação em entrada aleatória



Fonte: CSVs do experimento. Pontos: medianas; barras: mínimo e máximo das cinco medições, não intervalos de confiança. Eixos de tempo logarítmicos.

Tabela 4 Menores e maiores tempos com 100.000 elementos aleatórios

Operação	Menor mediana	Maior mediana
Construção (ms/lote)	Hash 5,24	Lista dupla 8.581,10
Busca presente (ns/op)	Hash 10,90	Lista dupla 94.700,00
Busca ausente (ns/op)	Hash 7,70	Lista dupla 191.379,90
Inserção (ns/op)	Hash 13,72	Lista dupla 178.945,87
Remoção (ns/op)	Hash 10,03	Lista dupla 83.607,45
Intervalo (ns/op)	Array ordenado 223,50	Lista dupla 351.576,50

Na Tabela 4, construção está em ms por lote; as demais operações estão em ns por operação. Para 100 mil elementos aleatórios, a hash registrou 13,72 ns por inserção e 10,03 ns por remoção. A lista registrou 178.945,87 e 83.607,45 ns/op. O custo da procura do valor está incluído nas remoções.

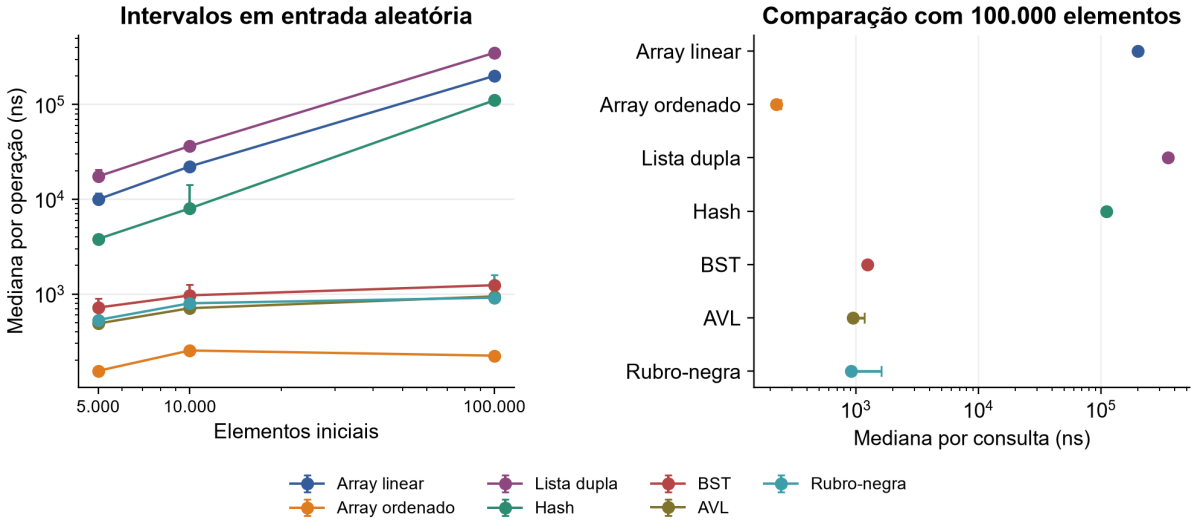
Comparando diretamente os arrays com 100 mil elementos aleatórios, o linear custou cerca de 84,75 vezes o ordenado na busca presente e 4,94 vezes na inserção. A busca binária também acelera a verificação de duplicatas do array ordenado, embora ele ainda precise deslocar elementos.

O array ordenado custou 4.463,62 ns por inserção e 3.720,19 ns por remoção. A posição é localizada por busca binária, mas deslocar os elementos mantém o custo linear dessas atualizações. O array linear não precisa preservar a ordem ao remover, porém sua busca sequencial elevou a remoção a 9.359,39 ns/op.

Entre as árvores balanceadas, a rubro-negra teve menores tempos de inserção e remoção que a AVL nas três ordens com 100 mil elementos. Na ordem aleatória, foram 162,75 contra 301,90 ns por inserção e 166,55 contra 299,22 ns por remoção. Essa vantagem foi observada nesta implementação; não foram medidos números de rotações para atribuí-la exclusivamente ao rebalanceamento.

4.5 Consultas por intervalo

Figura 5 Tempo por consulta de contagem em intervalos inclusivos



Fonte: CSVs do experimento. Pontos: medianas; barras: mínimo e máximo das cinco medições, não intervalos de confiança. Eixos de tempo logarítmicos.

O array ordenado apresentou a menor mediana de intervalo nas nove combinações de tamanho e ordem. Com 100 mil valores aleatórios, custou 223,50 ns por consulta. A rubro-negra registrou 914,50 ns e a AVL 944,00 ns, cerca de quatro vezes o tempo do array ordenado. A BST aleatória registrou 1.239,50 ns.

A diferença decorre da operação solicitada: contar, sem listar, os elementos entre dois limites. No array ordenado, duas buscas binárias localizam as fronteiras e a diferença das posições fornece a quantidade. O custo é O(log n). Nas árvores utilizadas, a consulta poda ramos pela ordenação, mas ainda visita os valores do intervalo, com custo O(h + k), em que h é a altura e k é o número de valores visitados dentro do intervalo.

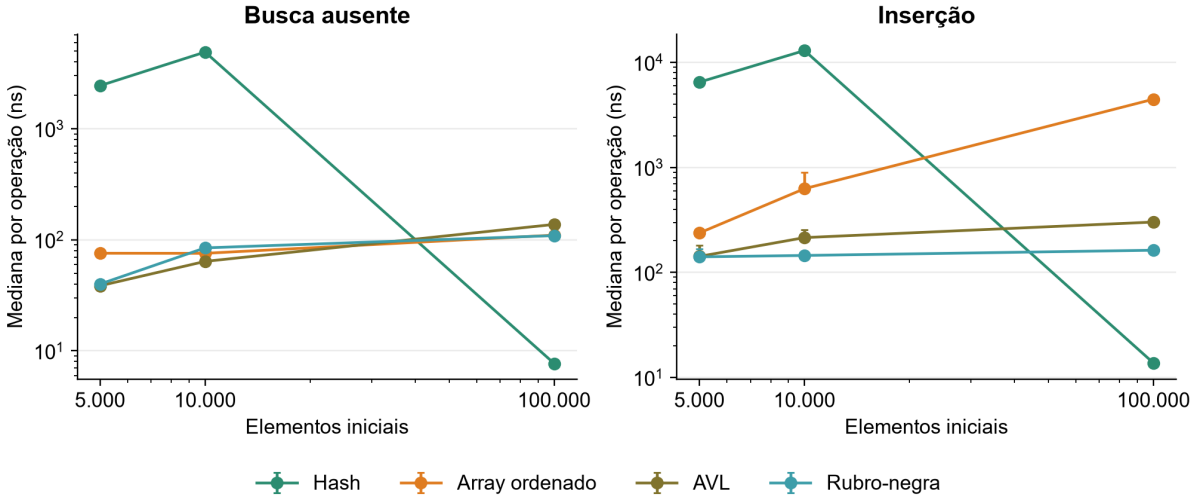
Como a largura do intervalo é fixa em 100, os conjuntos iniciais de pares fornecem apenas 50 ou 51 valores por consulta. A altura das árvores balanceadas influencia o caminho até o intervalo, mas a quantidade retornada praticamente não cresce com n. Na BST ordenada, a altura linear elevou a consulta para 197.117,00 ns na ordem crescente e 192.812,50 ns na decrescente.

A hash percorre sua capacidade para contar valores, enquanto array linear e lista percorrem o conjunto. Com 100 mil elementos aleatórios, registraram 110.878,00, 200.644,50 e 351.576,50 ns por consulta, respectivamente. A rapidez da hash nas buscas exatas não se transfere para intervalos. De 5 mil para 100 mil, a capacidade cresce 32 vezes e o tempo aleatório por intervalo 29,06 vezes, próximo do percurso O(capacidade) implementado.

Os tempos do array linear em intervalos também variaram com a ordem: 49.586,00 ns na crescente e 200.644,50 ns na aleatória. Em ambos os casos o método percorre n valores e continua O(n). A diferença também aparece nos conjuntos menores. Uma hipótese é a previsibilidade da condição de pertencimento ao intervalo nas entradas ordenadas. Não foram medidos desvios nem testada uma versão sem ramificações, portanto a causa permanece não confirmada.

4.6 Colisões e capacidade da tabela hash

Figura 6 Comportamento não monotônico da hash em entrada aleatória



Fonte: CSVs do experimento. Pontos: medianas; barras: mínimo e máximo das cinco medições, não intervalos de confiança. Eixos de tempo logarítmicos.

Tabela 5 Capacidade final da hash após a construção

n	Capacidade	Carga	Busca ausente ns/op	Inserção ns/op
5.000	8.192	61,04%	2.443,95	6.486,80
10.000	16.384	61,04%	4.907,15	12.971,00
100.000	262.144	38,15%	7,70	13,72

As capacidades da Tabela 5 foram deduzidas da política de crescimento do código. Com 5 mil e 10 mil valores, o maior par inicial excede a capacidade final. A função módulo faz pares diferentes disputarem os mesmos índices, e a sondagem linear pode formar blocos contíguos ocupados. Procurar um ímpar ausente exige atravessar esses blocos antes de encontrar uma posição vazia.

Com 100 mil elementos, a capacidade final é 262.144, maior que o maior par inicial, 200.000. Os pares passam a ocupar índices distintos, deixando posições ímpares livres. Isso explica, pela lógica da implementação, a queda da busca ausente de 4.907,15 ns/op em n = 10.000 para 7,70 ns/op em n = 100.000. A inserção também caiu de 12.971,00 para 13,72 ns/op. A explicação é baseada no código e no domínio das chaves; o experimento não registrou contadores de sondagens.

O caso de 100 mil é particularmente favorável: na tabela final, floorMod devolve o próprio valor e funciona como endereçamento direto para o domínio utilizado. Isso não elimina as colisões durante as etapas anteriores de redimensionamento na construção. A razão entre o domínio 2n e a capacidade, e não apenas o tamanho do conjunto, determina essa situação. Em n = 5.000 e ordem aleatória, a AVL foi mais rápida na busca ausente, e a BST foi mais rápida na inserção. A complexidade média esperada O(1) depende de condições de dispersão e não garante a menor mediana em cada conjunto.

4.7 Variação das medições e limites da comparação

As barras dos gráficos mostram o mínimo e o máximo das cinco medições, e não intervalos de confiança. A tabela abaixo permite distinguir medianas próximas de resultados instáveis. Sobreposição de faixas não é um teste de significância; diferenças pequenas são tratadas como descritivas, sem estabelecer superioridade universal.

Tabela 6 Variação em buscas presentes selecionadas em ns por operação

n e ordem	Estrutura	Mínimo	Mediana	Máximo
100.000 crescente	Hash	11,40	11,60	15,25
100.000 decrescente	Hash	38,10	42,95	58,80
100.000 aleatória	Hash	10,75	10,90	10,95
5.000 aleatória	AVL	35,85	36,10	49,55
5.000 aleatória	Rubro-negra	51,50	61,25	81,40
10.000 aleatória	AVL	57,60	58,25	74,25
10.000 aleatória	Rubro-negra	50,60	53,80	61,85
100.000 aleatória	AVL	96,45	98,30	110,10
100.000 aleatória	Rubro-negra	92,80	93,90	198,55

Com 100 mil elementos, a hash termina com o mesmo arranjo sem colisões nas três ordens. Ainda assim, a busca presente variou de mediana 10,90 ns na aleatória a 42,95 ns na decrescente. A faixa de 10,75–10,95 ns na aleatória não se sobrepõe a 38,10–58,80 ns na decrescente. Essa diferença entre cenários não vem de um layout final diferente e não foi explicada pela instrumentação disponível.

Na construção crescente, aumentar de 5 mil para 10 mil reduziu a mediana do array ordenado de 0,56 para 0,22 ms, da hash de 0,52 para 0,24 ms e da AVL de 1,21 para 0,87 ms. Complexidade assintótica não impede essa queda em tempos reais. Ela evidencia que um aquecimento e a ordem fixa não isolam JIT e efeitos do ambiente. Não foram refeitas medições com outro protocolo.

A execução não recebeu flags explícitas de heap ou coletor. O limite máximo de heap registrado foi 4.265.607.168 bytes; o coletor efetivo não foi registrado. Tomada, plano de energia e temperatura também não foram registrados e não podem ser reconstruídos com segurança. As chamadas pela interface podem ter custos de despacho ou otimizações diferentes, mas não houve inspeção do código compilado pela JVM para quantificar isso.

5 COMO ESCOLHER UMA ESTRUTURA

Para muitas buscas exatas, a hash é uma boa escolha quando a função de espalhamento e a capacidade distribuem bem as chaves. Com 100 mil valores aleatórios, teve 10,90 ns por busca presente e 7,70 ns por busca ausente. Esses números correspondem a uma tabela final sem colisões. A escolha exige avaliar o domínio das chaves e o espalhamento; a vantagem não pode ser atribuída ao volume maior por si só.

Para muitas inserções e remoções, sem necessidade de ordenar ou consultar intervalos, a hash também foi a mais rápida com 100 mil valores. Quando é necessário manter ordenação e comportamento logarítmico no pior caso, a rubro-negra foi a melhor opção entre as árvores balanceadas nas atualizações com esse tamanho. A AVL continua sendo uma alternativa, mas a comparação não sustenta afirmar que foi sempre superior nas buscas.

Para contar valores por intervalo em dados que mudam pouco, o array ordenado foi a escolha mais favorável: venceu as nove combinações avaliadas nessa operação. Quando atualizações frequentes precisam coexistir com intervalos, AVL ou rubro-negra evitam os deslocamentos do array. A escolha entre elas pode considerar o peso de cada operação, pois os tempos de intervalo ficaram próximos com 100 mil valores aleatórios.

6 CONCLUSÃO

Os resultados relacionam as diferenças de desempenho com o modo de armazenar e localizar valores. Array linear e lista apresentaram custos altos de busca e de construção com verificação de duplicatas. O array ordenado favoreceu buscas e contagem por intervalo, com custo de deslocamento nas atualizações. A BST foi competitiva na entrada aleatória, mas perdeu desempenho nas entradas ordenadas. AVL e rubro-negra evitaram essa degeneração.

A hash teve desempenho alto nas buscas exatas e atualizações com 100 mil elementos, em um arranjo final sem colisões. Os conjuntos de 5 mil e 10 mil mostram como a combinação da função módulo com o domínio pode alterar esse comportamento. Não há superioridade geral demonstrada para a hash. A principal aprendizagem é que a complexidade assintótica orienta a escolha, enquanto as medições mostram o efeito da implementação, da distribuição das chaves e do tipo de operação. Não há uma estrutura que tenha vencido todos os testes.

6.1 Limites da interpretação

Os dados representam uma rodada em uma máquina, com um aquecimento e cinco repetições por combinação. Os cenários foram executados em sequência fixa; compilação JIT, coleta de lixo, temperatura, cache e atividades do sistema podem influenciar as constantes. Não houve instrumentação para separar esses efeitos nem para contar rotações ou sondagens. Os resultados não provam complexidades ou diferenças universais entre implementações.

Fontes dos resultados

Dados: resultados/resultados_brutos.csv e resultados/resumo.csv. Ambiente: resultados/ambiente.txt. Conferência: resultados/validacao.txt e validar_resultados.py. Implementações: src. As Figuras 1 a 6 usam medianas e extremos das cinco repetições. As capacidades da hash foram calculadas a partir do código, sem nova medição.

APÊNDICE A RESULTADOS COM 5.000 ELEMENTOS

Medianas medidas. Construção em ms por lote; demais colunas em ns por operação. O menor valor de cada coluna está em negrito. Cada tabela compara o mesmo tamanho e a mesma ordem. Pres. significa busca presente e Aus. significa busca ausente.

Entrada crescente

Estrutura	Constr. ms	Pres. ns/op	Aus. ns/op	Inserção ns/op	Remoção ns/op	Intervalo ns/op
Array linear	2,54	862,05	881,80	988,40	535,60	2.218,00
Array ordenado	0,56	77,15	78,70	262,80	204,40	**148,50**
Lista dupla	24,10	4.196,65	9.261,90	8.698,00	4.725,40	9.609,50
Hash	**0,52**	**43,75**	2.650,90	6.286,40	**42,80**	3.726,00
BST	36,22	4.658,55	4.091,30	4.466,60	4.111,60	7.770,50
AVL	1,21	102,70	**43,65**	299,40	383,40	677,00
Rubro-negra	0,63	116,30	67,35	**134,00**	168,40	637,50

Entrada decrescente

Estrutura	Constr. ms	Pres. ns/op	Aus. ns/op	Inserção ns/op	Remoção ns/op	Intervalo ns/op
Array linear	2,73	510,75	987,75	1.045,60	402,20	2.127,50
Array ordenado	0,84	70,55	70,20	218,40	206,20	**143,50**
Lista dupla	20,51	4.724,50	9.319,35	8.411,60	4.285,20	10.301,50
Hash	**0,12**	**16,50**	2.530,00	6.056,60	**16,00**	3.696,50
BST	23,42	4.779,85	4.559,80	4.688,40	4.856,00	8.518,50
AVL	0,46	39,65	**44,50**	151,80	175,40	327,00
Rubro-negra	0,71	58,55	77,75	**116,20**	132,00	344,50

Entrada aleatória

Estrutura	Constr. ms	Pres. ns/op	Aus. ns/op	Inserção ns/op	Remoção ns/op	Intervalo ns/op
Array linear	2,65	434,85	891,75	966,60	446,60	9.992,50
Array ordenado	0,76	78,90	75,70	237,60	213,40	**154,50**
Lista dupla	21,66	4.852,95	9.284,25	8.560,60	3.851,00	17.440,00
Hash	**0,22**	**11,90**	2.443,95	6.486,80	**11,20**	3.815,00
BST	0,36	72,75	80,60	**87,80**	161,80	719,50
AVL	0,68	36,10	**38,55**	142,20	143,80	488,50
Rubro-negra	0,72	61,25	39,95	140,80	109,00	532,00

Fonte: resumo.csv. Valores arredondados somente para apresentação. O arquivo original contém seis casas decimais.

APÊNDICE B RESULTADOS COM 10.000 ELEMENTOS

Medianas medidas. Construção em ms por lote; demais colunas em ns por operação. O menor valor de cada coluna está em negrito. Cada tabela compara o mesmo tamanho e a mesma ordem. Pres. significa busca presente e Aus. significa busca ausente.

Entrada crescente

Estrutura	Constr. ms	Pres. ns/op	Aus. ns/op	Inserção ns/op	Remoção ns/op	Intervalo ns/op
Array linear	9,97	849,65	1.739,40	2.066,00	825,50	4.629,00
Array ordenado	**0,22**	76,35	75,20	386,30	339,80	**148,00**
Lista dupla	81,90	9.971,95	19.940,35	17.728,40	9.189,40	22.473,00
Hash	0,24	**12,55**	5.226,60	12.805,50	**13,20**	10.864,50
BST	112,12	9.292,10	8.839,15	11.315,10	11.673,20	18.270,00
AVL	0,87	66,30	**63,05**	216,20	208,00	337,00
Rubro-negra	0,86	66,25	70,10	**98,50**	126,50	450,00

Entrada decrescente

Estrutura	Constr. ms	Pres. ns/op	Aus. ns/op	Inserção ns/op	Remoção ns/op	Intervalo ns/op
Array linear	9,92	859,45	1.839,65	2.308,90	804,30	4.378,00
Array ordenado	3,27	76,30	**77,55**	406,70	340,80	**161,50**
Lista dupla	81,28	9.708,20	21.862,35	20.433,40	8.907,00	21.238,50
Hash	**0,26**	**12,70**	5.414,60	14.290,10	**11,60**	8.169,00
BST	101,78	10.480,30	10.146,10	10.144,50	10.552,70	16.679,00
AVL	2,07	141,60	78,55	369,70	359,30	878,00
Rubro-negra	1,05	127,60	99,40	**205,80**	217,40	1.167,50

Entrada aleatória

Estrutura	Constr. ms	Pres. ns/op	Aus. ns/op	Inserção ns/op	Remoção ns/op	Intervalo ns/op
Array linear	13,16	972,10	1.842,15	2.085,30	841,10	22.221,00
Array ordenado	2,16	76,90	75,55	627,40	553,10	**253,50**
Lista dupla	82,82	9.221,70	20.307,90	19.014,60	7.938,60	36.473,50
Hash	**0,52**	**11,25**	4.907,15	12.971,00	**11,20**	8.016,00
BST	0,92	91,55	103,85	147,50	85,30	967,50
AVL	1,76	58,25	**63,90**	214,60	206,00	708,50
Rubro-negra	0,86	53,80	84,60	**145,10**	167,30	799,00

Fonte: resumo.csv. Valores arredondados somente para apresentação. O arquivo original contém seis casas decimais.

APÊNDICE C RESULTADOS COM 100.000 ELEMENTOS

Medianas medidas. Construção em ms por lote; demais colunas em ns por operação. O menor valor de cada coluna está em negrito. Cada tabela compara o mesmo tamanho e a mesma ordem. Pres. significa busca presente e Aus. significa busca ausente.

Entrada crescente

Estrutura	Constr. ms	Pres. ns/op	Aus. ns/op	Inserção ns/op	Remoção ns/op	Intervalo ns/op
Array linear	1.180,83	9.469,60	20.135,85	24.037,34	9.700,05	49.586,00
Array ordenado	3,64	138,05	139,60	5.864,37	4.313,44	**228,00**
Lista dupla	9.175,76	98.002,05	206.653,25	195.713,79	88.428,59	220.209,50
Hash	**2,85**	**11,60**	**7,25**	**12,65**	**10,32**	111.336,00
BST	10.917,56	92.371,60	88.152,05	100.494,83	113.703,44	197.117,00
AVL	12,09	250,55	252,50	363,86	366,69	926,00
Rubro-negra	15,01	299,80	286,75	200,72	319,91	944,50

Entrada decrescente

Estrutura	Constr. ms	Pres. ns/op	Aus. ns/op	Inserção ns/op	Remoção ns/op	Intervalo ns/op
Array linear	1.090,65	10.299,00	20.948,35	31.503,31	10.605,52	50.894,50
Array ordenado	500,29	127,45	187,60	6.945,59	5.522,07	**297,50**
Lista dupla	10.580,15	104.848,45	197.998,35	189.673,24	95.163,79	206.488,00
Hash	**7,11**	**42,95**	**28,80**	**33,21**	**19,48**	132.468,00
BST	10.150,54	96.265,90	101.226,05	98.805,45	109.097,42	192.812,50
AVL	14,48	296,50	323,15	413,96	352,13	971,00
Rubro-negra	14,98	262,00	275,75	206,86	228,46	1.039,50

Entrada aleatória

Estrutura	Constr. ms	Pres. ns/op	Aus. ns/op	Inserção ns/op	Remoção ns/op	Intervalo ns/op
Array linear	1.022,72	8.691,70	20.195,30	22.041,51	9.359,39	200.644,50
Array ordenado	193,30	102,55	110,20	4.463,62	3.720,19	**223,50**
Lista dupla	8.581,10	94.700,00	191.379,90	178.945,87	83.607,45	351.576,50
Hash	**5,24**	**10,90**	**7,70**	**13,72**	**10,03**	110.878,00
BST	16,51	172,10	182,95	187,67	152,13	1.239,50
AVL	23,67	98,30	137,60	301,90	299,22	944,00
Rubro-negra	13,56	93,90	109,05	162,75	166,55	914,50

Fonte: resumo.csv. Valores arredondados somente para apresentação. O arquivo original contém seis casas decimais.

APÊNDICE D MENORES E MAIORES MEDIANAS COM 5.000 ELEMENTOS

Os extremos comparam as sete estruturas no mesmo tamanho, ordem e teste. Construção usa ms por lote; as demais linhas usam ns por operação. Os destaques são descritivos e não incorporam um teste de significância. Em caso de empate exato, os nomes são apresentados juntos.

Ordem	Operação e unidade	Menor	Tempo	Maior	Tempo
Crescente	Construção (ms/lote)	Hash	0,52	BST	36,22
Crescente	Busca presente (ns/op)	Hash	43,75	BST	4.658,55
Crescente	Busca ausente (ns/op)	AVL	43,65	Lista dupla	9.261,90
Crescente	Inserção (ns/op)	Rubro-negra	134,00	Lista dupla	8.698,00
Crescente	Remoção (ns/op)	Hash	42,80	Lista dupla	4.725,40
Crescente	Intervalo (ns/op)	Array ordenado	148,50	Lista dupla	9.609,50
Decrescente	Construção (ms/lote)	Hash	0,12	BST	23,42
Decrescente	Busca presente (ns/op)	Hash	16,50	BST	4.779,85
Decrescente	Busca ausente (ns/op)	AVL	44,50	Lista dupla	9.319,35
Decrescente	Inserção (ns/op)	Rubro-negra	116,20	Lista dupla	8.411,60
Decrescente	Remoção (ns/op)	Hash	16,00	BST	4.856,00
Decrescente	Intervalo (ns/op)	Array ordenado	143,50	Lista dupla	10.301,50
Aleatória	Construção (ms/lote)	Hash	0,22	Lista dupla	21,66
Aleatória	Busca presente (ns/op)	Hash	11,90	Lista dupla	4.852,95
Aleatória	Busca ausente (ns/op)	AVL	38,55	Lista dupla	9.284,25
Aleatória	Inserção (ns/op)	BST	87,80	Lista dupla	8.560,60
Aleatória	Remoção (ns/op)	Hash	11,20	Lista dupla	3.851,00
Aleatória	Intervalo (ns/op)	Array ordenado	154,50	Lista dupla	17.440,00

Fonte: resumo.csv. Esta tabela compara as medianas entre estruturas; a amplitude entre repetições está nos gráficos e na Tabela 6.

APÊNDICE E MENORES E MAIORES MEDIANAS COM 10.000 ELEMENTOS

Os extremos comparam as sete estruturas no mesmo tamanho, ordem e teste. Construção usa ms por lote; as demais linhas usam ns por operação. Os destaques são descritivos e não incorporam um teste de significância. Em caso de empate exato, os nomes são apresentados juntos.

Ordem	Operação e unidade	Menor	Tempo	Maior	Tempo
Crescente	Construção (ms/lote)	Array ordenado	0,22	BST	112,12
Crescente	Busca presente (ns/op)	Hash	12,55	Lista dupla	9.971,95
Crescente	Busca ausente (ns/op)	AVL	63,05	Lista dupla	19.940,35
Crescente	Inserção (ns/op)	Rubro-negra	98,50	Lista dupla	17.728,40
Crescente	Remoção (ns/op)	Hash	13,20	BST	11.673,20
Crescente	Intervalo (ns/op)	Array ordenado	148,00	Lista dupla	22.473,00
Decrescente	Construção (ms/lote)	Hash	0,26	BST	101,78
Decrescente	Busca presente (ns/op)	Hash	12,70	BST	10.480,30
Decrescente	Busca ausente (ns/op)	Array ordenado	77,55	Lista dupla	21.862,35
Decrescente	Inserção (ns/op)	Rubro-negra	205,80	Lista dupla	20.433,40
Decrescente	Remoção (ns/op)	Hash	11,60	BST	10.552,70
Decrescente	Intervalo (ns/op)	Array ordenado	161,50	Lista dupla	21.238,50
Aleatória	Construção (ms/lote)	Hash	0,52	Lista dupla	82,82
Aleatória	Busca presente (ns/op)	Hash	11,25	Lista dupla	9.221,70
Aleatória	Busca ausente (ns/op)	AVL	63,90	Lista dupla	20.307,90
Aleatória	Inserção (ns/op)	Rubro-negra	145,10	Lista dupla	19.014,60
Aleatória	Remoção (ns/op)	Hash	11,20	Lista dupla	7.938,60
Aleatória	Intervalo (ns/op)	Array ordenado	253,50	Lista dupla	36.473,50

Fonte: resumo.csv. Esta tabela compara as medianas entre estruturas; a amplitude entre repetições está nos gráficos e na Tabela 6.

APÊNDICE F MENORES E MAIORES MEDIANAS COM 100.000 ELEMENTOS

Os extremos comparam as sete estruturas no mesmo tamanho, ordem e teste. Construção usa ms por lote; as demais linhas usam ns por operação. Os destaques são descritivos e não incorporam um teste de significância. Em caso de empate exato, os nomes são apresentados juntos.

Ordem	Operação e unidade	Menor	Tempo	Maior	Tempo
Crescente	Construção (ms/lote)	Hash	2,85	BST	10.917,56
Crescente	Busca presente (ns/op)	Hash	11,60	Lista dupla	98.002,05
Crescente	Busca ausente (ns/op)	Hash	7,25	Lista dupla	206.653,25
Crescente	Inserção (ns/op)	Hash	12,65	Lista dupla	195.713,79
Crescente	Remoção (ns/op)	Hash	10,32	BST	113.703,44
Crescente	Intervalo (ns/op)	Array ordenado	228,00	Lista dupla	220.209,50
Decrescente	Construção (ms/lote)	Hash	7,11	Lista dupla	10.580,15
Decrescente	Busca presente (ns/op)	Hash	42,95	Lista dupla	104.848,45
Decrescente	Busca ausente (ns/op)	Hash	28,80	Lista dupla	197.998,35
Decrescente	Inserção (ns/op)	Hash	33,21	Lista dupla	189.673,24
Decrescente	Remoção (ns/op)	Hash	19,48	BST	109.097,42
Decrescente	Intervalo (ns/op)	Array ordenado	297,50	Lista dupla	206.488,00
Aleatória	Construção (ms/lote)	Hash	5,24	Lista dupla	8.581,10
Aleatória	Busca presente (ns/op)	Hash	10,90	Lista dupla	94.700,00
Aleatória	Busca ausente (ns/op)	Hash	7,70	Lista dupla	191.379,90
Aleatória	Inserção (ns/op)	Hash	13,72	Lista dupla	178.945,87
Aleatória	Remoção (ns/op)	Hash	10,03	Lista dupla	83.607,45
Aleatória	Intervalo (ns/op)	Array ordenado	223,50	Lista dupla	351.576,50

Fonte: resumo.csv. Esta tabela compara as medianas entre estruturas; a amplitude entre repetições está nos gráficos e na Tabela 6.